

VIETNAM AI STARS · 2026

Streamlet

Zero-ops real-time features.
Vibe-code your pipelines.

A single Rust binary. No Kafka, no Flink, no cluster, no platform team.
Scale = add memory. Author pipelines with the Claude skills that ship with it.

Hoang Phan

Founder & Engineering Lead · phan.minhhoang2606@gmail.com · github.com/petrpan26

THE OPPORTUNITY

Every startup wants real-time features. Nobody wants the Flink pager.

Fraud detection, live personalization, AI agent memory — the features that define a modern product can't wait for a nightly batch. Every growth-stage founder knows they need real-time. What stops them isn't budget or ambition — it's the operational burden. Running Kafka, Flink, Zookeeper, and a feature store means hiring people who've done it before, and those people are scarce, expensive, and already working at FAANG.

~50,000

Growth-stage startups globally that want real-time features

< 2%

That can staff a dedicated streaming-infra team today

4+

Systems to operate in the status-quo stack (Kafka, Flink, feature store, glue)

24/7

On-call rotation the day you turn Flink on in production

Real-time features are table stakes. Running a streaming platform is still a FAANG-only skillset.

THE PROBLEM

The real cost of real-time isn't the license. It's the pager.

To detect fraud, personalize a feed, or give an AI agent live context, you need features computed the instant an event lands — counts over the last hour, averages over the last day, distinct merchants in the last week. Every tool that does this was built for Uber and Netflix. Running them takes a team that most growth-stage startups can't staff:

THE STATUS-QUO STACK

Kafka + Zookeeper	event transport	brokers, rebalances
--------------------------	-----------------	---------------------

Flink / Spark Streaming	stateful compute	checkpoints, state TTL
--------------------------------	------------------	------------------------

Feature store (Feast/Tecton)	serving layer	online/offline sync
-------------------------------------	---------------	---------------------

ML platform engineer(s)	keep it all alive	24/7 on-call
--------------------------------	-------------------	--------------

4 systems. 3 engineers. A full-time pager rotation.

So what do growth-stage teams actually do?

- Run nightly batch pipelines — features are hours stale. Fraud already happened.
- Bolt rules onto Postgres triggers and cron jobs — fragile, invisible, impossible to test.
- Try to hire a streaming-infra specialist — takes six months, loses them to FAANG in twelve.
- Skip real-time entirely — leave revenue, safety, and retention on the table.

The gap is not talent. It's the operational burden.

Every growth-stage founder has hit this wall. Nobody has a clean answer yet.

THE SOLUTION

Streamlet: one binary, zero cluster, features in microseconds.

Streamlet is a single Rust binary you run on one machine. It speaks a custom TCP protocol: push an event, receive the updated features synchronously in the same round-trip. All state lives in memory. Periodic snapshots to local disk handle crash recovery.

Zero engineering ops

No Kafka, Flink, Zookeeper, or Redis sidecar. No platform team. No pager. Scaling model is trivial: when you need more, add RAM.

Dead-easy developer experience

One Python class is a production pipeline. No DAGs, no operators, no topology. A backend engineer ships their first feature in 10 minutes, not 10 days.

Vibe-codeable with Claude

Streamlet ships with Claude skills that author pipelines from plain-English specs. "Build me a fraud detector for card transactions" → valid, running code.

Cost to run in production: **\$20 – \$2,000 / month**. Scales from seed to Series C on the same binary.

Up to 100× cheaper than Kafka + Flink + feature store — and handles the same load up to 100K events/sec on one thread.

ARCHITECTURE

How it works — inside the binary

CLIENT

Python SDK

Declarative @st.stream definitions. Thin TCP client.

Any TCP client

Go, Node, Rust, C++, Java — custom binary protocol.

STREAMLET SERVER · SINGLE RUST BINARY

TCP protocol · PUSH / GET / SET / MSET / REGISTER

Length-prefixed binary frames over persistent connections.

Pipeline Engine

Operators (count, sum, avg, min, max, distinct_count, last). Expression evaluator. Cross-stream views. Lookups.

In-memory State Store

HashMap<EntityKey, FeatureMap>. Bucketed ring-buffer windows. HyperLogLog. TTL eviction.

Snapshot Persistence

Periodic serde+bincode dump to disk. Cooperative yielding. Load on startup for recovery.

OUT OF BAND

HTTP management API

Register pipelines, /metrics, /health, debug inspection.

Local disk snapshot

Versioned binary file. 30s default cadence. ~5s recovery / 1M keys.

Developer experience: features as code

```

# Declarative feature definitions. No DAGs.
import streamlet as st

@st.stream(key="user_id")
class Transactions:
    tx_count_1h      = st.count(window="1h")
    tx_sum_1h        = st.sum("amount", window="1h")
    unique_merchants = st.distinct_count("merchant_id",
                                         window="24h")

    failed_30m       = st.count(window="30m",
                               where="status == 'failed'")
    failure_rate      = st.derive("failed_30m / tx_count_1h")
    velocity_spike    = st.derive(
        "tx_count_1h / (tx_count_24h/24)")

# One round-trip. Event in, features out.
features = app.push(Transactions, {
    "user_id": "u_9182", "amount": 1_250_000,
    "merchant_id": "m_stripe_42", "status": "success"})

features.velocity_spike # 4.7 -> fraud engine blocks
features.failure_rate   # 0.12 -> elevated, flag review

```

Why this matters

- Feature definitions live next to application code, not in a separate YAML DSL or UI.
- Windows, filters, derived expressions, and cross-stream lookups in the same declarative block.
- The Python SDK never touches the hot path — it serializes definitions once and the Rust server does the rest.
- Same API a backend engineer at a Series B fintech or a solo founder at a seed company can pick up in 10 minutes.
- Claude skills that ship with Streamlet turn plain-English specs into valid `@st.stream` classes — no learning curve.

What makes it fast — the core technical bets

- 1 Single-threaded core**
Like Redis. One thread, zero locks, zero contention. Predictable latency under load. Key-partitioned multi-threading is a drop-in upgrade for v2 because keys are independent.
- 2 Bucketed sliding windows**
Every windowed operator uses a fixed ring buffer of buckets. $O(\text{window/bucket})$ memory per key — a few KB regardless of event rate. On read: sum the non-expired buckets in a single tight loop.
- 3 HyperLogLog for distinct count**
Unique-merchant, unique-IP and unique-device counts in 12KB per key, forever, at any cardinality. Well-understood error bounds (~1%). Bounded memory is what makes zero-ops possible.
- 4 Server-side expression evaluator**
`derive` and `where` expressions are parsed at register-time into an AST and evaluated in Rust at event-time. Python never runs on the hot path — the guarantee that matters for latency.
- 5 Snapshot-based durability (Redis RDB model)**
Periodic serialize-to-disk with cooperative yielding. No WAL, no LSM tree. Losing 30 seconds of state on crash is an acceptable trade for an order-of-magnitude simplicity win.
- 6 Cross-stream views and cross-key lookups**
`tx_count / login_count` across streams, or `lookup(merchant.chargebacks, on='merchant_id')` from a user stream. Views recompute on read — no pre-materialization needed.

BENCHMARKS

Performance targets — sized for one machine

100K+

events / sec sustained on one thread

< 100 μ s

PUSH latency (p99, single-event)

< 50 μ s

GET latency (p99)

< 5 KB

Memory per key (10 mixed features)

Plus: < 1s snapshot write per 1M keys · < 5s recovery from disk on startup · single-binary deployment

HOW WE GET THERE · DESIGN CHOICES THAT MAKE THESE NUMBERS ACHIEVABLE

- In-memory HashMap lookups (no disk I/O on the hot path) — same class as Redis.
- Fixed-size ring buffer per operator — event handling is a branch and an increment, not an allocation.
- Custom length-prefixed binary protocol over persistent TCP — no HTTP header overhead per event.
- Single-threaded event loop — zero lock contention, cache-friendly, latency is a flat line under load.

USE CASES

One binary, four high-revenue verticals

Fintech fraud & risk

Velocity, device, merchant, and geo features for every card authorization or payment. Think Stripe Radar, but as a primitive every payments startup can afford.

1 instance handles 50M txns/day

E-commerce personalization

Live session signals for Shopify Plus, Series B marketplaces, and DTC brands — time on category, cart abandonment, last-viewed product — all sub-ms.

Feature store without the feature store

AI agent memory & context

Every LLM agent startup needs rolling per-user state: last N intents, entity frequency, tool-call history, anomaly scores — returned in a single GET call.

The missing primitive for 2026 AI products

Ad-tech & real-time bidding

Per-user frequency capping, budget pacing, and audience features at bid-request latency (< 10 ms) without a Flink cluster and a four-person SRE rotation.

Sub-10ms bid-time feature serving

IMPACT

3 engineers → 0. Two weeks to first pipeline → two minutes.

STATUS QUO

Kafka + Flink + Feature store

3 engineers · 2 weeks

- Streaming-infra specialist required
- 24/7 on-call pager rotation
- Weeks to stand up the first feature
- Expertise is scarce and non-transferable

WITH STREAMLET

One Rust binary

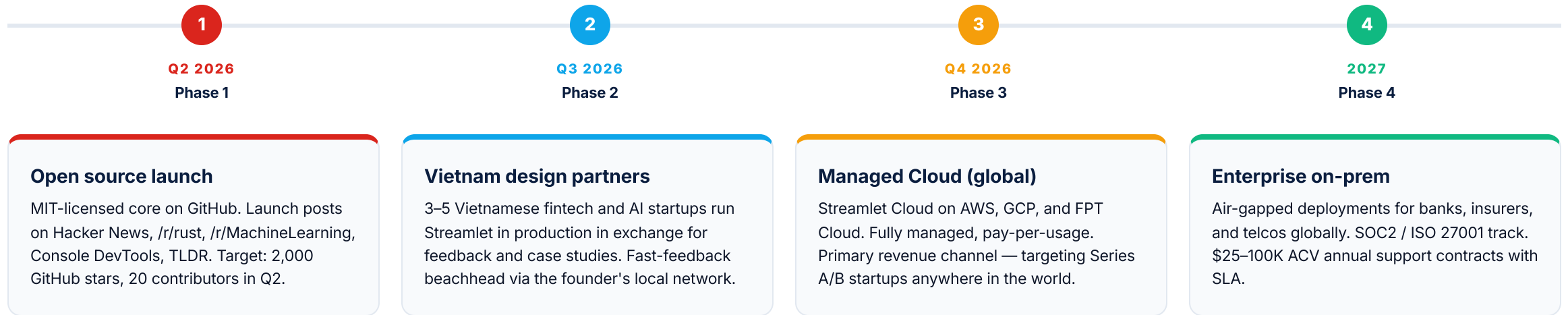
0 engineers · 2 minutes

- Any backend engineer, no specialist hire
- No pager — restart = full recovery
- Pipeline authored by Claude from a prompt
- Scale by adding RAM, not headcount

If 5,000 growth-stage startups adopt Streamlet over 5 years:

- ~15,000 engineer-years redirected from platform ops into product
- Real-time fraud, personalization, and AI-agent memory become standard, not luxury
- Vietnam becomes the origin of a globally-used developer primitive — our VAS contribution

Four-phase rollout: OSS-first, Vietnam beachhead, global target



Project status — core engine is real

SHIPPED · IN THE REPO TODAY

Core state store

HashMap<EntityKey, FeatureMap> with TTL eviction, live + static features.

Bucketed sliding windows

Ring buffer implementation powering count, sum, avg, min, max.

HyperLogLog operator

Distinct count with bounded 12KB memory per key.

Expression parser + evaluator

Server-side AST for derive / where with arithmetic and booleans.

TCP protocol + server

Length-prefixed binary frames. PUSH, GET, SET, MSET, REGISTER.

Snapshot persistence

serde+bincode periodic dumps. Cooperative yielding. Recovery on startup.

Cross-stream views & lookups

derive() across streams, lookup() across keys.

Claude skills for pipeline authoring

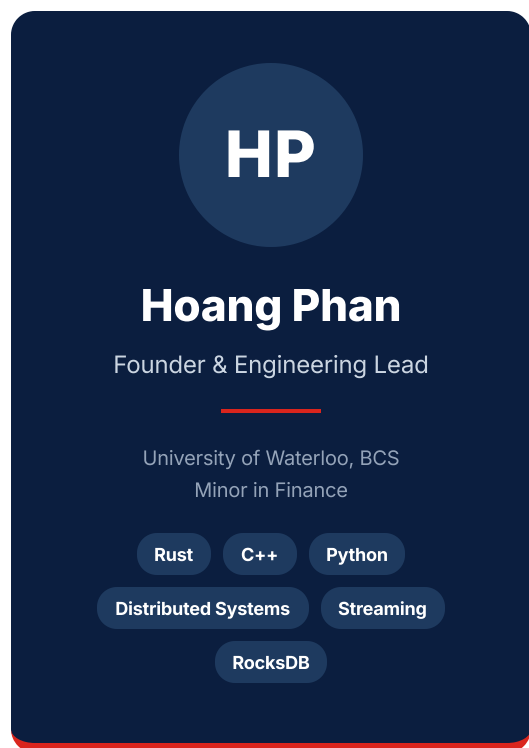
Shipped: Claude writes valid @st.stream pipelines from plain-English specs.

NEXT · COMPETITION TIMELINE

- | | |
|---------|--|
| Q2 2026 | Python SDK v1 + first design partner deploy |
| Q2 2026 | Incremental snapshots (phase 9, in planning) |
| Q3 2026 | Benchmark harness + public perf report |
| Q3 2026 | Open source launch + Vietnamese docs |
| Q4 2026 | Managed Cloud private beta on FPT Cloud |
| Q4 2026 | 10 production deployments milestone |

TEAM

Built by someone who built this before



A dark blue profile card with rounded corners and a red bottom border. At the top is a dark blue circle with the white letters 'HP'. Below it, the name 'Hoang Phan' is written in white, followed by 'Founder & Engineering Lead' in a smaller font. A thin red horizontal line is below the title. Underneath, it says 'University of Waterloo, BCS' and 'Minor in Finance'. At the bottom, there are five rounded rectangular buttons: 'Rust', 'C++', 'Python', 'Distributed Systems', and 'Streaming', with 'RocksDB' centered below them.

HP

Hoang Phan
Founder & Engineering Lead

University of Waterloo, BCS
Minor in Finance

Rust C++ Python
Distributed Systems Streaming
RocksDB

Fennel AI · acquired by Databricks (2024–2025)

Distributed Systems Engineer on the exact same class of system Streamlet is. Re-architected streaming join and window operators for 10× runtime and memory efficiency. Designed approximate percentile data structures that landed 5 enterprise customers.

Viggle AI · Engineering Lead, Toronto (2025–present)

Leading a 4-engineer team building real-time inference infrastructure. Reduced time-to-first-chunk from 8s to 2s, cut \$1M+ in annual cloud spend, scaled the office from 1 to 8 engineers.

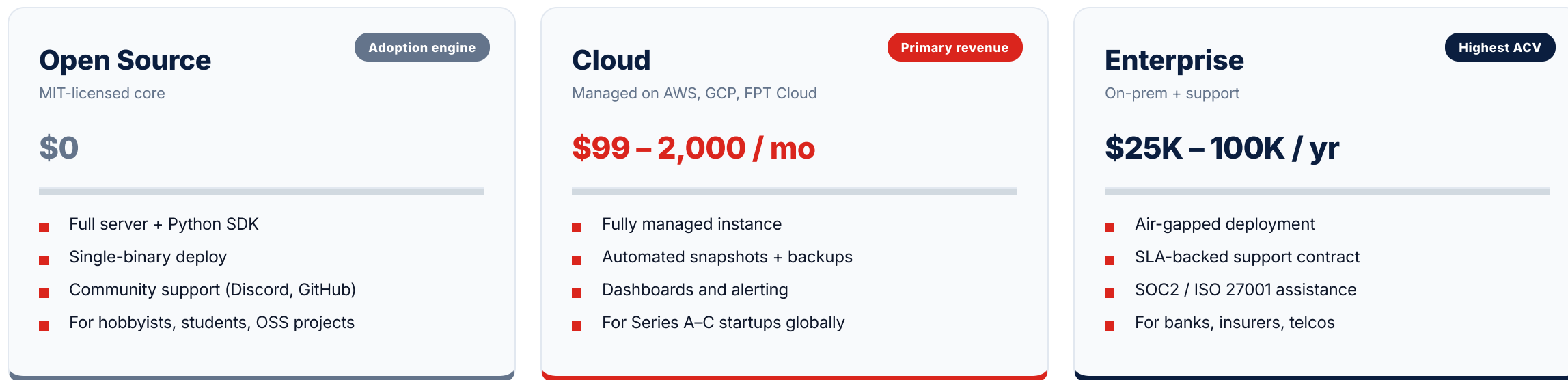
Faire · ML Platform Engineer (2022–2023)

Built the self-serve deep-learning deployment path, cut Snowflake spend 3× via incremental transformations, built fraud-review product-quality signals that raised manual-review precision by 10%.

Google · SWE Intern (2021) · APIO Silver Medal (2nd in Vietnam)

Ranked 35/164 across 30 countries at the Asia-Pacific Informatics Olympiad. Vietnamese-origin founder with a decade of systems and competitive programming foundation.

Open-core: adoption first, revenue second



Comparable open-core companies (same shape, different primitive):

Redis Labs — \$2B valuation · **ClickHouse** — \$2B valuation · **Confluent (Kafka)** — public, \$7B market cap

Streamlet is to real-time features what Redis is to in-memory data.

THE VISION

Zero-ops, AI-authored real-time infrastructure.

Streamlet is the default real-time feature primitive for the next 50,000 growth-stage startups — one binary to run, plain-English specs to author. Built out of Vietnam, used everywhere.

WHAT WE ARE ASKING FOR

Mentorship

Guidance from VAS judges and mentors on taking a Vietnam-origin developer-tools startup to a global audience.

Design partners

Warm introductions to 3–5 growth-stage fintech, e-commerce or AI startups — Vietnamese or global — willing to be first production users.

Seed capital

A small pre-seed round to fund a second engineer and launch the managed-cloud beta.

</content> </invoke>

Cảm ơn — Thank you.

phan.minhhoang2606@gmail.com · github.com/petrpan26